Compiladores e Intérpretes

Tema 4.1 – Analizador Semántico – Introducción

Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
2026

Intuiciones Análisis Semántico

- La **tarea final** del compilador es **traducir** el **programa fuente** en un código que se pueda **ejecutar directamente** en la maquina destino
- Para esto **necesita conocimiento del programa** que va mas allá de la **estructura sintáctica**.

Intuiciones Análisis Semántico

- Las tareas del **analizador semántico**, en general para un lenguaje de programación con **tipado estático**, se pueden dividir en **dos**:

Entender y controlar todas las entidades declaradas
Chequeo de Declaraciones

Entender y controlar todas las sentencias asociadas a las entidades recolectadas
Chequeo de Sentencias

Intuiciones Análisis Semántico

- En general, el **analizador semántico** realiza estos **controles** una vez que el **sintáctico finalizo** sus tareas

Estos controles son efectuados sobre una **representación alternativa (intermedia)** del fuente, donde se **recolectaron** y **estructuraron** las **entidades** declaradas y sus **sentencias**

- Esta **representación se construye** a medida que se realiza el **análisis sintáctico**



Intuiciones Análisis Semántico

- En general, el **analizador se** …e el **sintáctico finalizo** sus tar…

Existe otra alternativa que es implementar los controles semánticos mientras se realiza el análisis sintáctico… Sin embargo, esto ya **no se realiza en los compiladores/interpretes modernos** por que no permite referencias hacia adelante

Estos controles son efectua… **va (intermedia)** del fuente, de… **entidades** declaradas y sus **sentencias**

- Esta **representación se construye** a medida que se realiza el **análisis sintáctico**



Intuiciones Análisis Semántico

- Entonces antes de arrancar con los controles tenemos que estudiar la forma de estas estructuras y como vamos a construirlas
- Para esto ultimo, es necesario realizar **acciones especiales** dentro del **analizador sintáctico**

Las **Gramáticas de Atributos** son las herramientas formales utilizadas para **diseñar** estas acciones en el **analizador sintáctico**

Compiladores e Intérpretes

Tema 4.2 – Analizador Semántico – Esquemas de Traducción

Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
2026

Gramáticas de Atributos - Introducción

- Son gramáticas libres de contexto **aumentadas** con conjunto de **reglas** (**acciones**) que **especifican computaciones**
  - Estas **acciones** son asociadas a **producciones**

TipoPrimitivo → **String** {//Accion para devolver tipo String}
TipoPrimitivo → **int** {//Accion para devolver tipo int}

Gramáticas de Atributos - Introducción

- Son gramáticas libres de contexto **aumentadas** con conjunto de **reglas** (**acciones**) que **especifican computaciones**
  - Estas **acciones** son asociadas a **producciones**
- Las **acciones** pueden usar a **atributos**
  - Los **atributos** están **asociados** a los **símbolos de la producción**

Gramáticas de Atributos - Introducción

- Son gramáticas libres de contexto **aumentadas** con conjunto de **reglas** (**acciones**) que **especifican computaciones**
  - Estas **acciones** son asociadas a **producciones**
- Las **acciones** pueden usar a **atributos**
  - Los **atributos** están **asociados** a los **símbolos de la producción**

TipoPrimitivo → **String** {TipoPrimitivo.tipo = *tipoString*}
TipoPrimitivo → **String** {TipoPrimitivo.tipo = *tipoint*}

Gramáticas de Atributos - Introducción

- Son gramáticas libres de
  de **reglas** (**acciones**) que
  - Estas **acciones** son a
- Las **acciones** pueden usar a **at**
  - Los **atributos** están **asociados** **símbolos de la producción**

Que **significa** que ejecutar esta acción?
Como influye en **TipoPrimitivo**?
Cuando se **Ejecuta**?

TipoPrimitivo → **String** {TipoPrimitivo.tipo = *tipoString*}
TipoPrimitivo → **String** {TipoPrimitivo.tipo = *tipoint*}

Esquemas de Traducción

Esquemas de Traducción - Introducción

Los **esquemas de traducción (EDT)** nos permiten **embeber** las **acciones** dentro de las producciones para **indicar explícitamente ese orden**

- En un **EDT** vamos a tener producciones, por ejemplo, de la forma:

$$A \rightarrow B \ \{C.y = B.x\} \ C \ \{A.z = C.w\}$$

Esquemas de Traducción - Introducción

Los **esquemas de traducción (EDT)** nos permiten **embeber** las **acciones** dentro de las producciones para **indicar explícitamente ese orden**

- En un **EDT** vamos a tener producciones, por ejemplo, de la forma:

$$A \rightarrow B \ \{C.y = B.x\} \ C \ \{A.z = C.w\}$$

Se ejecutan siguiendo una derivacion a izquierda. Es decir, primero resolvemos B, despues ejecutamos la primer accion, depues C y por ultimo la segunda accion

Atributos de Salida (Sintetizados)

- Un atributo es **de Salida** en toda asignación que le **asignemos valor**, esta asociado el **NT** de la **izquierda** la **producción**

$$A \rightarrow B \ \{C.y = B.x\} \ C \ \{A.z = C.w\}$$

Atributos de Entrada (Heredados)

- Un atributo es **de Entrada** si le asignamos un **valor** solo cuando aparece asociado al **símbolo** de la **parte derecha** de la **producción**.

$$A \rightarrow B \ \{C.y = B.x\} \ C \ \{A.z = C.w\}$$

Código en Producciones

- Las **acciones** pueden ser sean **porciones de código** en un lenguaje bien definido (por ej. Java)

```
VarLocal → Tipo idMV    {   if(!tabla.almacenado(idMV.lex))
                                tabla.guardar(idMV.lex, Tipo.tipo)
                            else ERROR!
                        }
```

- Aun así **solo** podemos referirnos a **atributos asociados** a la **producción** a la cual la acción esta vinculada.
  - Pero, en caso de ser **necesario** podemos usar **variables globales**.

Ejemplo

D → T L
T → **idC** | **String** | **int**
L → **idMV** R
R → **idMV** R | e

Gramática para declarar variables

Queremos guardar el nombre de cada id con su respectivo tipo, usando la función Guardar(nombre, tipo)
Si importar si estaba o no almacenado

Esquemas de Traducción Implementación

EDTs en Analizadores Sintácticos

- En los **analizadores descendentes predictivos recursivos**, la implementación de una **EDT es directa**
- ¿Cómo **codificamos** las **acciones**?
  - Cuando codificamos la **parte derecha**, también **codificamos las acciones** siguiendo el orden en el que aparecen en la producción
- ¿Cómo modelamos los **atributos de Salida**?
  - Son **retornados** por el subprograma correspondiente al **NT asociado al atributo**
- ¿Cómo modelamos los **atributos de Entrada**?
  - Son los **parámetros de entrada** correspondiente al **NT asociado al atributo**

EDTs en Analizadores Sintácticos

- Por ejemplo:

A → r B C

A → r B {C.y = B.x} C {A.z = C.w}

```
void A(){
    match("r")
    B()
    C()
    }
```

Implementación EDT

```
A(){
    match("r")
    x = B()
    w = C(x)
    return w}
```

Esquemas de Traducción

- **No** cualquier gramática de atributos **tiene un esquema de traducción** asociado

